Volume 15 – Exchange Integration & Adapter Framework

AI Quant Trading Platform

Version: 1.0

Document Type: Exchange Integration & Adapter Framework

Classification: Core Integration Layer

---

1. Purpose

This document defines the architecture, standards, interfaces, workflows, and implementation guidelines for integrating cryptocurrency exchanges into the AI Quant Trading Platform.

The objective is to provide a unified interface that allows the platform to communicate with multiple exchanges without changing the core trading engine.

---

2. Design Goals

The Exchange Integration Layer shall:

- Support multiple exchanges
- Provide a unified API
- Hide exchange-specific implementation details
- Handle exchange failures gracefully
- Support REST and WebSocket APIs
- Normalize data formats
- Support sandbox and production environments
- Enable plug-and-play exchange adapters

---

3. Supported Exchanges

Phase 1

- Mudrex
- Binance
- Bybit

Phase 2

- OKX
- KuCoin
- Bitget
- Gate.io
- MEXC

Future

- Coinbase
- Kraken
- Hyperliquid
- Deribit

---

4. Architecture

```
Trading Engine
      |
Exchange Interface
      |
_____________________________

Mudrex Adapter

Binance Adapter

Bybit Adapter

OKX Adapter

KuCoin Adapter

Bitget Adapter

_____________________________
REST + WebSocket Clients
      |
Exchange APIs
```

---

5. Adapter Responsibilities

Each adapter must implement:

- Authentication
- Connection Management

- Balance Retrieval
- Position Retrieval
- Order Placement
- Order Cancellation
- Order History
- WebSocket Streaming
- Market Data
- Error Translation
- Rate Limit Handling

---

6. Standard Exchange Interface

Every adapter must support:

Connection

- connect()
- disconnect()
- ping()

Market Data

- getMarkets()
- getTicker()
- getOrderBook()
- getCandles()
- getTrades()

Account

- getBalance()
- getPositions()
- getOpenOrders()
- getTradeHistory()

Orders

- placeOrder()
- cancelOrder()
- modifyOrder()
- closePosition()

Streaming

- subscribeTicker()
- subscribeTrades()
- subscribeOrderBook()
- subscribeOrders()
- subscribePositions()

---

7. Authentication

Support:

- API Key
- API Secret
- Passphrase (where required)

Credentials must be encrypted and never exposed to clients.

---

8. Connection Manager

Responsibilities

- Connection Pooling
- Heartbeat
- Auto Reconnect
- Timeout Detection
- Failover
- Connection Monitoring

---

9. REST API Layer

Responsibilities

- Market Data
- Historical Data
- Account Data
- Orders
- Positions

Retry transient failures using configurable backoff.

---

10. WebSocket Layer

Responsibilities

- Live Prices
- Order Updates
- Balance Updates
- Position Updates
- Funding Rate
- Liquidation Events

Features

- Heartbeats
- Auto Reconnect
- Subscription Recovery

---

11. Data Normalization

All exchange-specific responses must be converted into a common internal format.

Examples:

Market

- Symbol
- Base Asset
- Quote Asset

Orders

- Order ID
- Status
- Side
- Type
- Quantity
- Price

Balances

- Asset
- Available
- Locked
- Total

---

12. Order Lifecycle

Created

↓

Validated

↓

Submitted

↓

Accepted

↓

Filled

↓

Partially Filled

↓

Closed

↓

Archived

Every adapter must map native exchange statuses into the common lifecycle.

---

13. Order Types

Support where available:

- Market
- Limit
- Stop
- Stop Limit
- OCO
- Trailing Stop

Adapters should report unsupported types clearly.

---

14. Position Synchronization

Sync:

- Entry Price
- Current Price
- Quantity
- Unrealized PnL
- Margin
- Leverage
- Liquidation Price (if applicable)

Synchronization intervals are configurable.

---

15. Balance Synchronization

Track:

- Available Balance
- Locked Balance
- Wallet Balance
- Margin Balance
- Unrealized PnL

---

16. Market Data Synchronization

Collect:

- OHLCV
- Trades
- Order Book
- Funding Rates
- Open Interest

Cache frequently accessed data to reduce API usage.

---

17. Error Handling

Translate exchange-specific errors into standard platform errors.

Examples

- Authentication Failed
- Insufficient Balance
- Invalid Order
- Rate Limit
- Maintenance
- Symbol Not Found
- Network Error

---

18. Rate Limit Management

Each adapter maintains:

- Request Counters
- Cooldown Periods
- Retry Policies
- Queue Management

Avoid exceeding exchange limits.

---

19. Sandbox Support

Adapters should support:

- Test Environment
- Paper Trading
- Mock Exchange

Production and testing credentials must remain separate.

---

20. Exchange Health Monitoring

Monitor:

- API Availability
- Latency
- WebSocket Status
- Authentication Status
- Error Rate

Generate alerts for persistent failures.

---

21. Security

- Encrypt API credentials
- Never log secrets
- Rotate credentials when required
- Validate exchange permissions
- Restrict access by tenant and user

---

22. Plugin Model

Adding a new exchange should require:

1. Implement the standard interface.
2. Register the adapter.
3. Configure credentials.
4. Run integration tests.
5. Enable for users.

Core business logic must remain unchanged.

---

23. Performance Requirements

- Efficient connection reuse
- Configurable concurrency
- Minimal latency for market updates
- Asynchronous I/O
- Graceful degradation during outages

---

24. Testing Requirements

Each adapter must include:

- Unit Tests
- Integration Tests
- Sandbox Validation
- Error Handling Tests
- Reconnection Tests
- Rate Limit Tests

No adapter may be released without passing certification tests.

---

25. Future Enhancements

- Multi-exchange smart routing
- Cross-exchange arbitrage
- Liquidity aggregation
- Best execution routing
- Automatic failover between exchanges
- Unified portfolio across exchanges

---

26. Design Principles

- One interface, many exchanges.
- Exchange-specific complexity remains inside adapters.
- All adapters follow identical contracts.
- Trading engines never communicate directly with exchange APIs.
- Every exchange action is auditable.
- Adapters are independently testable and deployable.